Q1:

a. From a physical perspective, the sensor size plays a huge role in determining the number of megapixels the camera can take. For example, a large sensor can pack large number of pixels, making the resolution higher. Not forgetting about the display medium as well, larger screens needs higher resolution images (such as 4k).

   From a hardware standpoint, storage space and more powerful processors are needed when it comes to higher resolution images to handle the larger file sizes.

   From a software aspect, efficient compression algorithms (In time and space) are necessary to manage large file sizes without compromising image quality and to avoid losing data.

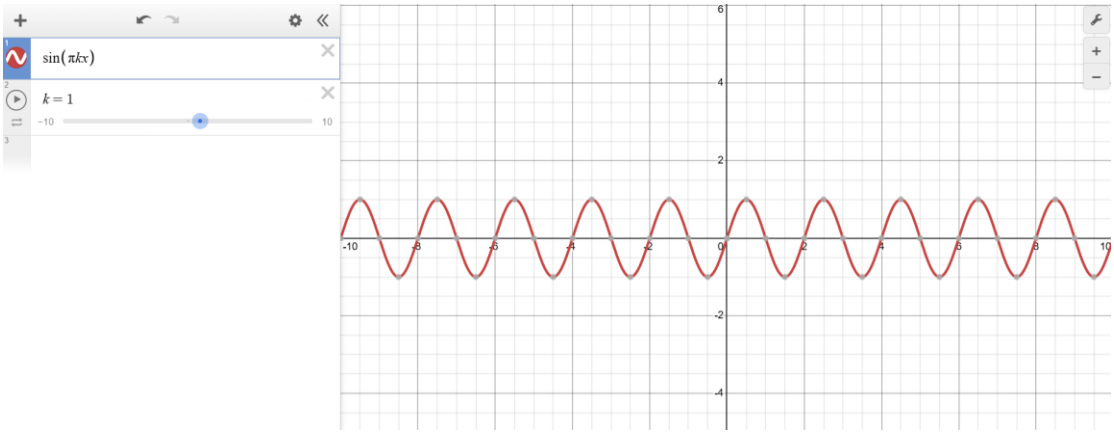
b. Some considerations that go into deciding how strong the quantization of an image is:

   Resolution Compatibility: Older displays had lower resolutions, so the smaller details provided by high quantization would not be visible on the screen. While today's displays with high dynamic range (HDR) and deep color require finer quantization to take advantage of their capabilities.

   Storage and bandwidth: high quantization increases file size because more information are required to represent the image which leads to occupying more space. Furthermore, sending high quantization images over slow networks will cause delays. While lower quantization makes transmission and processing faster.

   Performance: Early computers lacked the processing power to handle high quantization efficiently. To handle this, they reduced the quantization so they can be capable of processing the data.

Q2:



a. As we know, the wavelength is the distance over which the pattern repeats itself. For the given function $\sin(\pi kx)$ for $k = 1$ the distance over which the pattern repeats itself is 2. For $k = 2$ the distance is 1 (which is quite obvious since when we increase the k, the function will be squeezed which make the distance smaller by k times). Hence, in general we'll get that the wavelength is $\frac{2}{k}$.

b. To fully restore the image without aliasing we need to ensure that the frequency f_{sample} of the sampling grid is at least 2 times larger than the frequency f_I of the image. Mathematically:

$$f_{sample} \geq 2f_I$$

From the previous section, we concluded that $f_I = \frac{k}{2}$ (the reciprocal of the wavelength).
In the grid, the width of the columns is A and the distance between 2 columns is A as well. Therefore, the distance over which the pattern repeats itself is $2A$. Meaning, the wavelength is $2A$. Hence $f_{sample} = \frac{1}{2A}$.
We want to find k such that

$$f_{sample} = \frac{1}{2A} \geq k = 2f_I$$

1. For $A = 0.25$, we get that:

$$f_{sample} = \frac{1}{0.5} \geq k = 2f_I \Longrightarrow k \leq 2$$

Therefore, for each $k \leq 2$ we can restore the image without aliasing.

2. For $A = 2$, we get that:

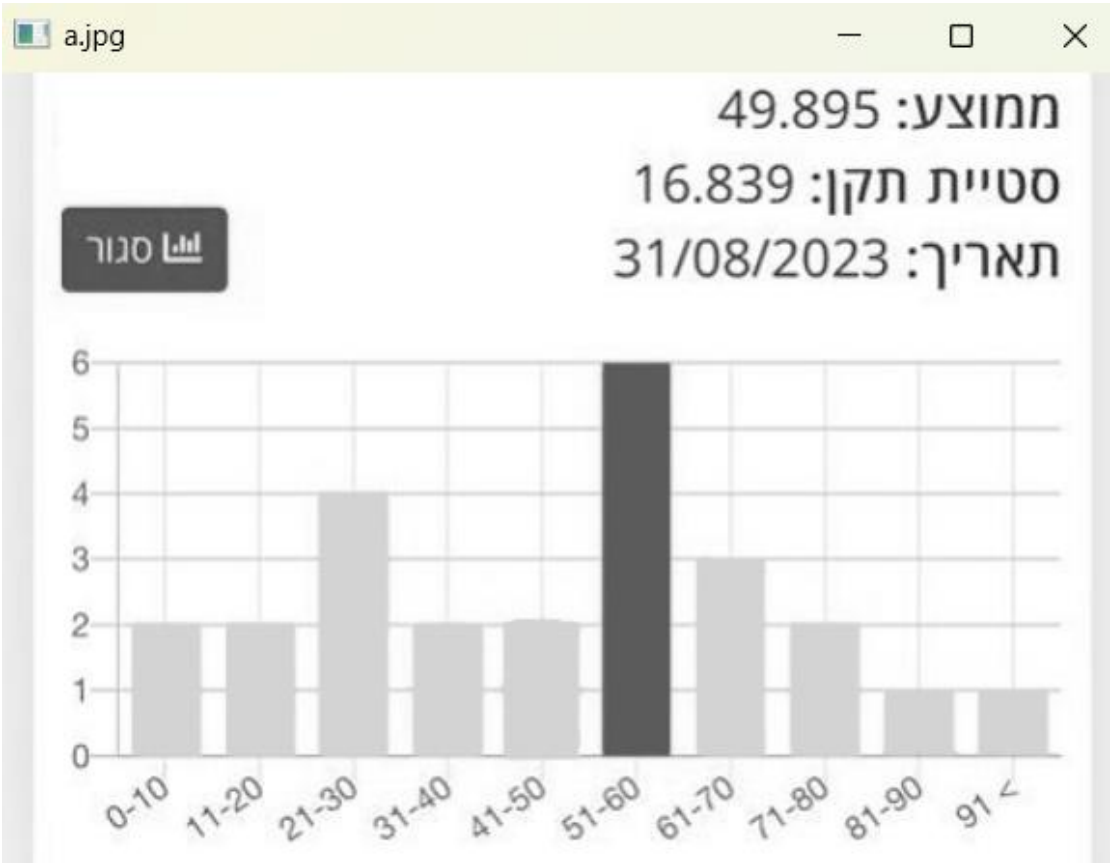
$$f_{sample} = \frac{1}{4} \geq k = 2f_I \Longrightarrow k \leq \frac{1}{4}$$

Therefore, for each $k \leq \frac{1}{4}$ we can restore the image without aliasing.

Report part:

We'll explain step by step what we did throughout the code to make everything crystal clear.

In sections a, b, we loaded the data and the numbers using read_dir and displayed the images on the screen by CV2.imshow method, and got the following results:



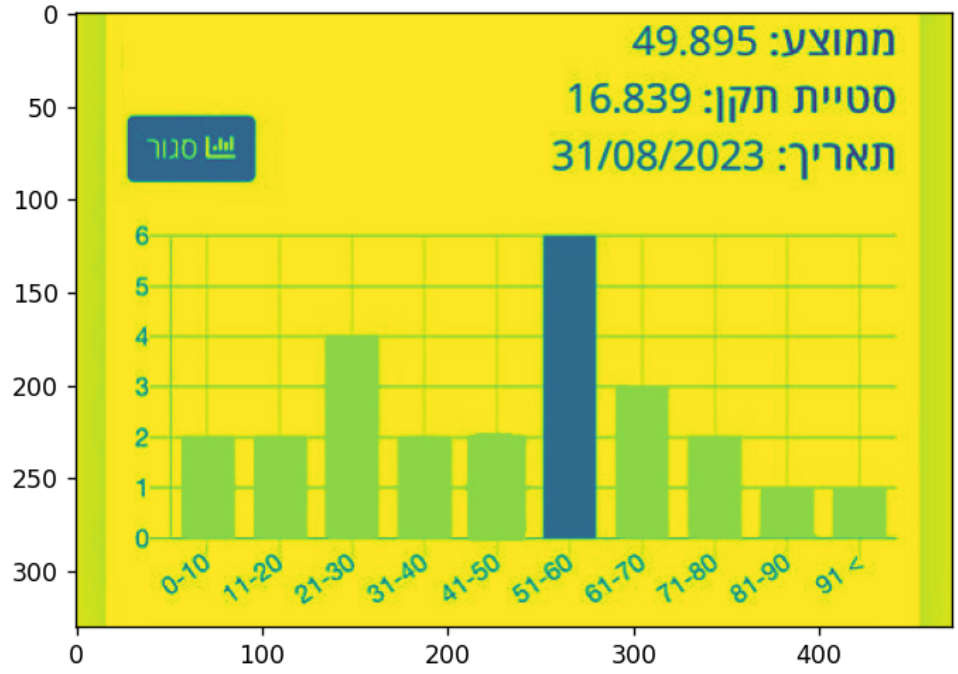
And

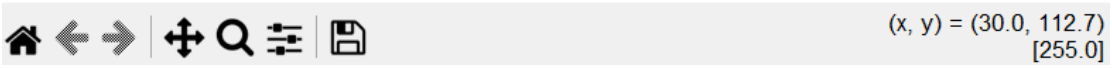


Our task now was to find the highest number along the vertical axis of the histogram. To do so, we realized that we could hardcode the task since all the images/numbers have the same format. Hence, we displayed the grade histograms on the screen using "metaplot" library (we used this library because it easily enables us to determine in which window to look for in the histograms) via the code:

```
plt.imshow(images[0])
plt.show()
```

Which gives the following output:





If we take a look at the bottom right corner, we can see $(x, y) = (30,112.7)$. this is the window of the highest number on the vertical access which is what we're looking for.

Now, since we know the location of the target window, we can procced to the next step.

We preprocessed the data to make the code faster by calculating the cumulative histogram for both the numbers array and the window at position (30,113) by:

```
for j in range(len(images)):
    windows = np.lib.stride_tricks.sliding_window_view(images[j], number_shape)
    target_window = windows[113, 30]
    windows_cum_hist.append(calc_cum_hist(target_window))

for i in range(len(numbers)):
    numbers_cum_hist.append(calc_cum_hist(numbers[i]))
```

At this point, we're ready to compare the cumulative histogram. We did so by calculating the EMD of the number and the window and decide if they match using the threshold value 260.

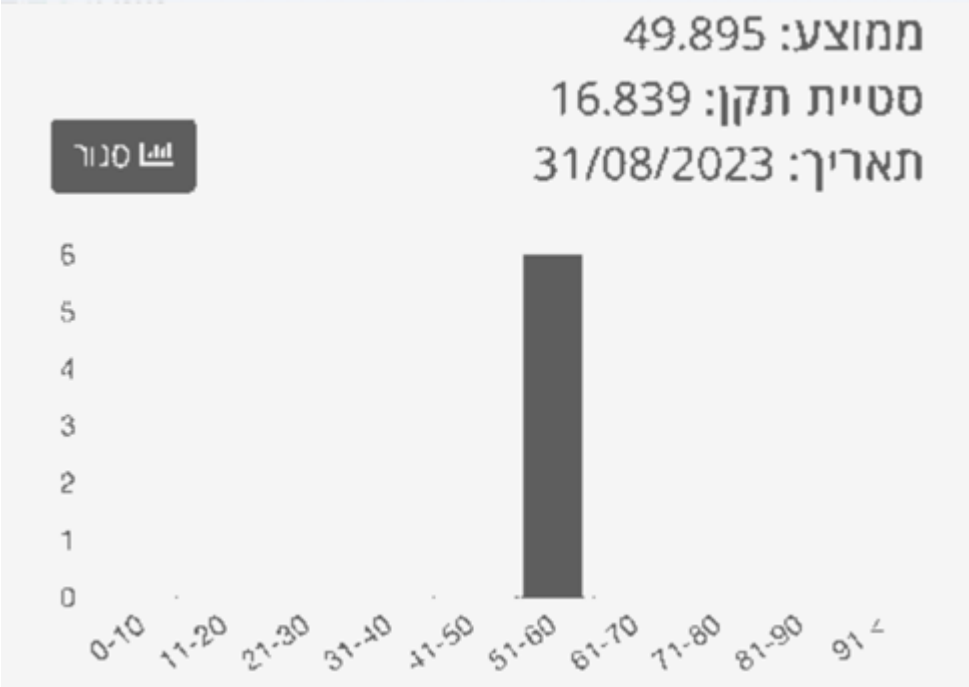
```
def compare_hist(target_window_cum, target_cum_hist):
    emd = np.sum(np.abs(target_window_cum - target_cum_hist))
    return emd < 260
```

After implementing compare_hist function, section d was straightforward and gave:

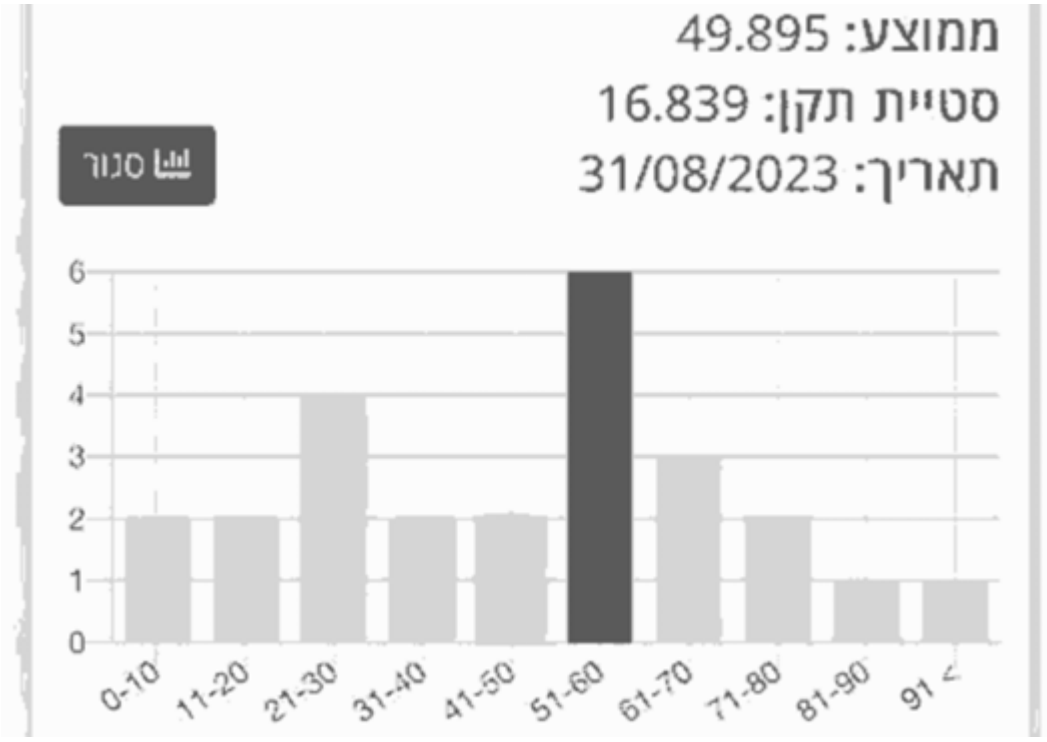
```
the highest column in image a.jpg contains: 6 students.
the highest column in image b.jpg contains: 6 students.
the highest column in image c.jpg contains: 1 students.
the highest column in image d.jpg contains: 6 students.
the highest column in image e.jpg contains: 5 students.
the highest column in image f.jpg contains: 4 students.
the highest column in image g.jpg contains: 9 students.
```

Next, we quantize the images with 3 colors (because in 2 colors only the dark blue bar will appear and that's clearly not enough, hence we came up with the conclusion that if we make all the bars dark blue, then quantization with 2 colors may work).

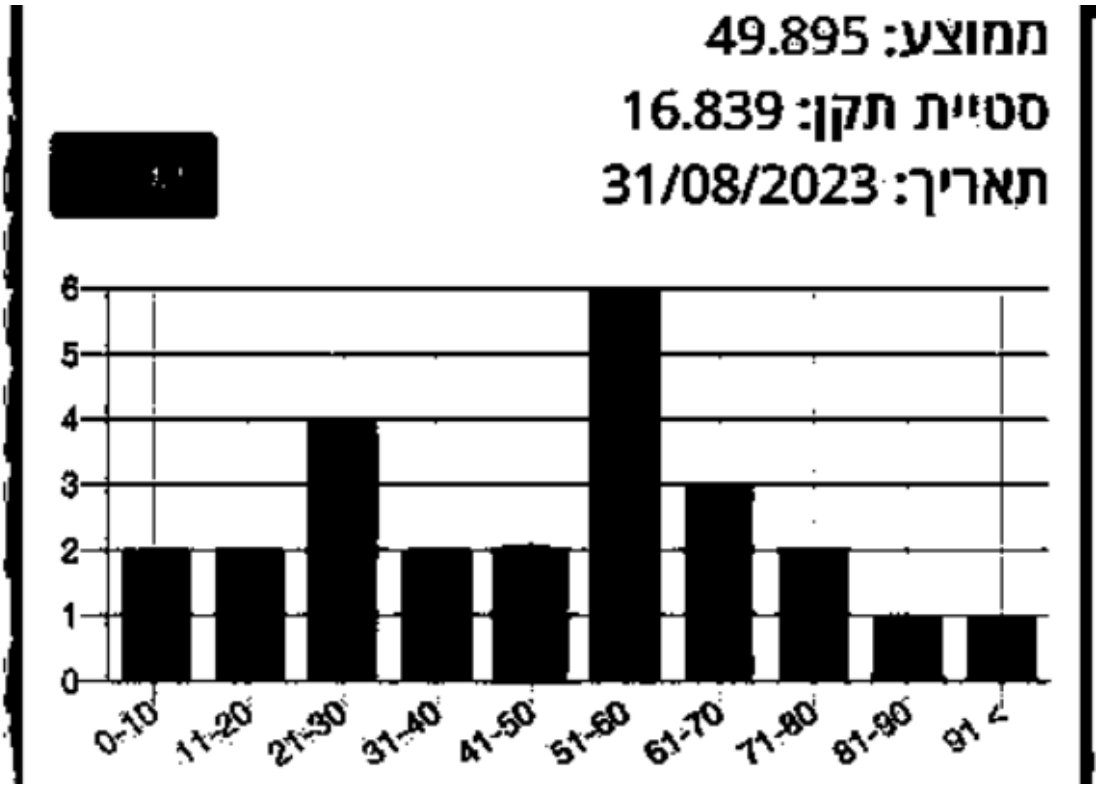
For reference, this is how the image will look like after quantization with 2 colors:



And this with 3 colors (there is a little bit of noise on the sides, but it doesn't bother):



After the quantization, we convert the images to black and white. Like this:



We set the threshold to 230 because it worked fine.

Worth noting that we change the 1 in get_bar_height to 0 because the bars are black.

So, after running the function on the first image we got:

```
the bar number 0 in a.jpg has height: 48
the bar number 1 in a.jpg has height: 48
the bar number 2 in a.jpg has height: 102
the bar number 3 in a.jpg has height: 48
the bar number 4 in a.jpg has height: 49
the bar number 5 in a.jpg has height: 156
the bar number 6 in a.jpg has height: 75
the bar number 7 in a.jpg has height: 48
the bar number 8 in a.jpg has height: 20
the bar number 9 in a.jpg has height: 20
```

Notice that the highest number is 156 (which is the height of the maximum number of students on the vertical axis in pixels).

With this information, we can conclude what is the number of students per bin using this line:

```
bar_heights.append(round(max_student_list[im_index] * bar_height / 156))
```

Then print all.

```
Histogram a.jpg gave 2,2,4,2,2,6,3,2,1,1
Histogram b.jpg gave 6,2,1,1,3,3,6,2,2,3
Histogram c.jpg gave 0,0,0,0,0,0,1,1,1,1
Histogram d.jpg gave 1,0,2,3,4,3,5,5,6,2
Histogram e.jpg gave 2,1,1,3,2,5,1,1,2,3
Histogram f.jpg gave 1,0,1,1,1,4,1,1,2,1
Histogram g.jpg gave 1,1,1,3,1,2,9,3,3,0
```